TASK 3:

Introduction to Numpy, Pandas and Matplotlib



Team RoboManipal 2026

Numpy:

Q1)Vectorization(Calculating MSE):

- Create a 1D array of 100 actual target values (`y_true`) using random numbers between 0 and 50. Create another 1D array of 100 predicted values (`y_pred`) using random numbers between 0 and 50.
- Calculate the Mean Squared Error (MSE) between these two arrays **without using any `for` loops**.

$$MSE = \frac{1}{n}\sum(y_true - y_pred)^2$$

Q2)Broadcasting(Centering using mean of data):

- Generate a 150×4 2D array (representing 150 samples and 4 features) with random integers between 10 and 100. Calculate the mean of each of the 4 features. Then, subtract this mean from every single sample in the dataset.
- Do this in a single mathematical operation by relying on NumPy broadcasting. Do not use loops to iterate through the 150 rows.

Q3)Boolean Indexing(Masking):

- Create a 5×5 matrix containing random numbers drawn from a standard normal distribution, which will contain both positive and negative values.
- Use boolean masking (indexing) to replace all negative numbers in the matrix with 0, leaving the positive numbers completely unchanged.
- Apply different normalization techniques on the positive numbers

Q4)Reshaping and Dimensionality:

- Create a 1D array of 3,072 random integers ranging from 0 to 255 (similar to raw pixel values of an image).

- Reshape this flat array into a 3D matrix with the shape (32, 32, 3) representing a single 32×32 RGB image. Then, extract just the "Red" channel (the entire 32×32 grid where the last dimension's index is 0).

Q5)Normalization:

- Imagine you have test scores from 3 different subjects for 50 students. Create a 50×3 matrix representing these scores using random integers between 40 and 100. Standardize the dataset by calculating the Z-score for each subject (column) independently.
- Apply the Z-score formula $Z = \frac{X-\mu}{\sigma}$ (where μ is the mean and σ is the standard deviation) using NumPy broadcasting. You must not use any for loops.
- Figure out where normalization techniques are used and apply other normalization techniques such as min-max,decimal-scaling etc.

Pandas and Matplotlib:

You are given a dataset(Palmer's Penguin data), the general description of each column is given below. Use pandas for all the data cleaning and manipulation tasks and use matplotlib for the data visualization tasks.

Dataset Key Columns:

- **`species`**: Penguin species (*Adelie*, *Chinstrap*, or *Gentoo*)
- **`island`**: Island in Antarctica where the penguin was found (*Biscoe*, *Dream*, or *Torgersen*)
- **`bill_length_mm`**: Length of the beak in millimeters (numeric value)
- **`bill_depth_mm`**: Depth of the beak in millimeters (numeric value)
- **`flipper_length_mm`**: Length of the flipper in millimeters (numeric value)
- **`body_mass_g`**: Weight of the penguin in grams (numeric value)
- **`sex`**: *MALE* or *FEMALE*

Q1)Data Exploration:

Load the dataset from the URL. Display the first **5 rows.** Then, print the **structural summary** of the DataFrame to find out the **total number of rows, data types of each column, and exactly which columns are missing data.** Finally, generate the basic statistical summary (**mean, min, max, quartiles**) for all numeric columns.(You may use functions such as `df.head()`, `df.describe()` etc)

Q2)Handling missing values:

- Only 2 penguins are completely missing physical measurements (like `body_mass_g`). Drop the rows where `body_mass_g` is missing.
- About 9 remaining penguins are missing their `sex`. Fill those missing values in the `sex` column with the most common value (or simply `'UNKNOWN'`).
- Figure out when it is appropriate to drop a whole row(data sample) vs replacing missing values.
- You may use functions like df.dropna() and df.fillna() for the task

Q3)Feature Engineering and Data Transformation:

- Create a new column called `body_mass_kg` by dividing `body_mass_g` by `1000`.
- Create a column called `bill_ratio` by dividing `bill_length_mm` by `bill_depth_mm`.
- Use binning to categorize penguins into a new column called `size_category` based on their `body_mass_g`: `'Small'` (2500–3500g), `'Medium'` (3500–4500g), and `'Large'` (4500–6500g).
- Use direct column math to perform the arithmetic operations, do not use for loops.
- Use pandas.cut() function for binning the penguins into different categories

Q4)Data Filtering:

- Find all **Gentoo** penguins (`species == 'Gentoo'`) that live on **Biscoe** island and weigh more than **5,000 grams**. Display only their `species`, `sex`, `flipper_length_mm`, and `body_mass_g`, sorted from heaviest to lightest.
- Use boolean masking to get the correct set of penguins and standard pandas functions for locating and sorting the columns

Q5)Data Grouping, Aggregation and One-Hot Encoding:

1. Count how many penguins belong to each `species`.
2. Group the dataset by `species` and `sex`, and calculate the average `flipper_length_mm` and `body_mass_g` for each group.
3. Convert the categorical `island` column into one-hot encoded numerical columns (0s and 1s). You will learn more about this type of encoding and its use in the next task.

Q6)Data Visualization:

- **Bar Chart(Aggregation by Groups):** Group the dataset by `species`, calculate the average `body_mass_g` for each species, and plot the result as a bar chart with an appropriate title.
- **Scatter Plot (Feature Relationship):** Create a scatter plot with `flipper_length_mm` on the x-axis and `body_mass_g` on the y-axis to visualize whether penguins with longer flippers weigh more.
- **Histogram (Data Distribution):** Draw a histogram of the `bill_length_mm` column using 15 bins to see the distribution of beak lengths across all penguins.